

Encapsulamento em UML e Java

Encapsulamento em UML

O encapsulamento é um dos pilares fundamentais da orientação a objetos e, nesta parte do módulo, vamos explorar como ele é representado no UML. Em resumo, o encapsulamento é a prática de esconder os detalhes internos de uma classe e expor apenas o que é necessário para que outras partes do sistema interajam com ela. Isso garante maior controle sobre os dados e comportamentos da classe, promovendo segurança e flexibilidade no código.

No diagrama de classes do UML, o encapsulamento é representado por meio da visibilidade dos atributos e métodos. Atributos privados (identificados pelo símbolo "-") são acessíveis apenas dentro da própria classe, enquanto métodos públicos (com o símbolo "+") estão disponíveis para outras classes. Vamos ver um exemplo:

Exemplo: Classe “ContaBancaria”

A classe “ContaBancaria” contém dois atributos encapsulados:

- **saldo** (double) e **numeroDaConta** (int), ambos privados.

Além disso, a classe oferece métodos públicos:

- **depositar(double valor)** e **sacar(double valor)** para operações externas,
- **getSaldo()** e **getNumeroDaConta()** para acessar os valores dos atributos.

Neste exemplo, as outras classes podem interagir com “ContaBancaria” apenas por meio de seus métodos públicos, sem acessar diretamente os atributos encapsulados. Essa estrutura permite que a implementação interna dos métodos possa mudar sem afetar outras partes do sistema.

Benefícios do Encapsulamento

Proteção: os detalhes de implementação são mantidos privados, evitando alterações indevidas de fora da classe.

Facilidade de manutenção: como os detalhes internos são escondidos, mudanças na implementação não afetam as interfaces usadas por outras classes.

Na próxima etapa, usaremos o Astah para modelar essa classe e gerar o código Java, aplicando o conceito de encapsulamento tanto no diagrama quanto no código gerado.

Criação de classes no Astah

Nesta parte do módulo, vamos aplicar o conceito de encapsulamento na criação de classes utilizando a ferramenta Astah. A ferramenta permite que desenhemos diagramas de classes e configuremos atributos e métodos de acordo com os princípios da orientação a objetos.

Vamos utilizar como exemplo uma classe chamada “Aluno”. Nessa classe, definimos alguns atributos básicos como:

nome (String)

matrícula (int)

curso (String)

idade (int)

Todos esses atributos são privados, ou seja, não podem ser acessados diretamente de fora da classe. Esse é o primeiro aspecto fundamental do encapsulamento: ocultar os detalhes internos da implementação. Para permitir o acesso controlado a esses atributos, utilizamos métodos públicos **getter** e **setter**.

Exemplo de Métodos Get e Set

getNome(): retorna o nome do aluno.

getMatrícula(): retorna a matrícula.

getCurso(): retorna o curso.

getIdade(): retorna a idade.

Cada um desses métodos oferece acesso controlado aos atributos, sem expor diretamente os dados internos da classe. Para modificar esses atributos, utilizamos os métodos set:

setNome(String nome): define o nome do aluno.

setMatrícula(int matrícula): define a matrícula.

setCurso(String curso): define o curso.

setIdade(int idade): define a idade.

Outros Métodos

Além dos métodos get e set, podemos incluir ações específicas que a classe “Aluno” pode realizar, como:

estudar()

fazerProva()

Esses métodos representam ações que o objeto “Aluno” pode executar, sem a necessidade de alterar diretamente os atributos.

Aplicando o Encapsulamento no Astah

Ao criar o diagrama de classes no Astah, você definirá a visibilidade dos atributos como **privada** (símbolo “-”) e dos métodos como pública (símbolo “+”). Isso garante que os atributos estejam encapsulados e que o acesso a eles ocorra apenas por meio dos métodos definidos.

Você também pode configurar diferentes níveis de visibilidade, como protected (protegido) ou **package** (visibilidade de pacote), mas, para respeitar

o princípio do encapsulamento, a melhor prática é manter os atributos privados e acessar seus valores apenas através dos métodos públicos.

Essa abordagem garante maior segurança e flexibilidade no código, permitindo que as classes interajam entre si de forma controlada e eficiente.

Encapsulamento em Java

O encapsulamento é um dos princípios fundamentais da orientação a objetos, e em Java, ele é alcançado ao declarar os atributos de uma classe como privados e fornecer métodos públicos para acessar e modificar esses atributos. Esses métodos são conhecidos como getters (para obter valores) e setters (para alterar valores).

Exemplo Prático: Classe “Funcionário”

Vamos usar como exemplo uma classe Java chamada “Funcionário”. Nela, temos três atributos:

nome (String)

cargo (String)

salário (double)

Esses atributos são definidos como **privados**, o que impede que outras classes acessem diretamente esses dados. No entanto, para permitir que outras partes do programa interajam com esses atributos, fornecemos métodos **getter** e **setter**.

Implementação em Java

Aqui está como seria a implementação do encapsulamento na classe “Funcionário” em Java:

```
public class Funcionario {  
  
    private String nome;
```

```
private String cargo;
```

```
private double salario;
```

```
// Getter e Setter para o atributo nome
```

```
public String getNome() {
```

```
    return nome;
```

```
}
```

```
public void setNome(String nome) {
```

```
    this.nome = nome;
```

```
}
```

```
// Getter e Setter para o atributo cargo
```

```
public String getCargo() {
```

```
    return cargo;
```

```
}
```

```
public void setCargo(String cargo) {
```

```
    this.cargo = cargo;
```

```
}
```

```
// Getter e Setter para o atributo salario
```

```
public double getSalario() {
```

```
    return salario;
```

```
}
```

```
public void setSalario(double salario) {
```

```
        this.salario = salario;

    }

}
```

Entendendo o Encapsulamento

No exemplo acima, os atributos nome, cargo e salário estão encapsulados, ou seja, são inacessíveis diretamente fora da classe. A manipulação desses atributos é feita exclusivamente por meio dos métodos públicos get e set. Isso garante que qualquer modificação ou consulta aos dados ocorra de maneira controlada, respeitando o princípio do encapsulamento.

Por exemplo, se quisermos acessar ou modificar o nome de um funcionário, utilizaremos os métodos **getNome()** e **setNome(String nome)**. Dessa forma, os detalhes de implementação (como os nomes e tipos de variáveis) estão ocultos, garantindo maior flexibilidade e segurança no código.

Vantagens do Encapsulamento em Java

Segurança: os atributos privados impedem o acesso indevido aos dados.

Controle: o uso de métodos públicos permite validar e controlar as mudanças nos atributos.

Manutenção: facilita a modificação interna da classe sem impactar outras partes do sistema.

Ao aplicar o encapsulamento corretamente, garantimos que as classes em Java sejam mais robustas e fáceis de manter.

Geração de código Java no Astah

Nesta última parte do módulo, vamos aprender como gerar código Java automaticamente a partir de um diagrama de classes criado no Astah. O Astah é uma ferramenta poderosa que permite modelar classes em UML e, com apenas alguns cliques, gerar o esqueleto do código-fonte Java, facilitando o processo de desenvolvimento.

Geração de Código a Partir de Diagrama de Classes

Vamos utilizar como exemplo duas classes que criamos no Astah: Aluno e Curso.

Na classe Curso, definimos os seguintes atributos:

nome (String)

créditos (int)

disciplinas (int)

Esses atributos foram encapsulados como privados. Além disso, criamos dois métodos:

getCurso(), que retorna o nome do curso;

setCurso(String nome), que define o nome do curso.

Passo a Passo para Exportar Código Java no Astah

1 - Criar o diagrama de classes: No Astah, criamos o diagrama das classes desejadas, no caso “Aluno” e “Curso”.

2 - Exportar o código Java: No menu do Astah, vá em Tools > Java > Export Java. Selecione o diretório onde deseja salvar o código gerado e as classes que deseja exportar.

3 - Abrir o código em uma IDE: Após exportar, podemos abrir o código gerado em uma IDE como o Android Studio. O código gerado conterá o esqueleto da classe, com os atributos privados e os métodos públicos get e set.

Exemplo do Código Gerado

Aqui está um exemplo do código Java gerado para a classe Curso:

```
public class Curso {  
  
    private String nome;
```

```
private int creditos;
```

```
private int disciplinas;
```

```
public String getNome() {
```

```
    return nome;
```

```
}
```

```
public void setNome(String nome) {
```

```
    this.nome = nome;
```

```
}
```

```
public int getCreditos() {
```

```
    return creditos;
```

```
}
```

```
public void setCreditos(int creditos) {
```

```
    this.creditos = creditos;
```

```
}
```

```
public int getDisciplinas() {
```

```
    return disciplinas;
```

```
}
```

```
public void setDisciplinas(int disciplinas) {
```

```
    this.disciplinas = disciplinas;
```

```
}
```



```
}
```

Este código foi gerado automaticamente a partir do diagrama de classes no Astah, poupando tempo e esforço na escrita manual. Os atributos estão devidamente encapsulados, e os métodos get e set permitem acesso controlado aos dados.

Integração com IDEs

Uma das vantagens de usar ferramentas como o Astah em conjunto com uma IDE, como o Android Studio, é a automação do processo de desenvolvimento. Você pode criar os diagramas, gerar o código Java automaticamente e, em seguida, completar os detalhes de implementação diretamente na IDE.

Além disso, o processo também funciona ao contrário: se você já tiver o código-fonte Java, é possível importar esse código para o Astah e gerar os diagramas de classes a partir dele. Isso facilita a manutenção e evolução do projeto, permitindo que o código e os diagramas fiquem sincronizados.

Conteúdo Bônus

Para se aprofundar no tema desta aula, sugiro que assista ao vídeo intitulado “Encapsulamento em 10 minutos”, que está disponível no canal DevSuperior no YouTube.

Referência Bibliográfica

ASCENCIO, A. F. G.; CAMPOS, E. A. V. de. Fundamentos da programação de computadores. 3.ed. Pearson: 2012.

BRAGA, P. H. Teste de software. Pearson: 2016.

GALLOTTI, G. M. A. (Org.). Arquitetura de software. Pearson: 2017.

GALLOTTI, G. M. A. Qualidade de software. Pearson: 2015.

MEDEIROS, E. Desenvolvendo software com UML 2.0 definitivo. Pearson: 2004.

MORAIS, I. S. de. (Org.). Engenharia de software. Pearson: 2017.

PFLEEGER, S. L. Engenharia de software: teoria e prática. 2.ed. Pearson: 2003.

SOMMERVILLE, I. Engenharia de software. 10.ed. Pearson: 2019.